FDE ACADEMY · LESSON 4 OF 9

Tool-Using Agents

Tools that fail on purpose, and an agent loop that can’t run forever.

Raunak Sharan
Enterprise Agentic AI Engineering

git checkout lesson-04

OBJECTIVES

By the end of today you can

01

Wrap every external check so it returns a typed result, never None

02

Inject timeouts, 429s, 500s and bad payloads on demand

03

Decide which checks are required and what each failure does

04

Give an agent an allow-list of tools with validated arguments

05

Bound the loop with a step cap and a deadline

06

Fall back to a human with a machine-readable reason

TOOLS FAIL

A tool call is a network call

Timeout

The bureau never answers. You still owe the applicant a response.

429

Rate-limited, with a retry-after. Not an outage; not yet a retry either.

500

The vendor is down. Their incident is now your decision.

Malformed

It answered: score “seven hundred”. The most dangerous one, because it looks like success.

The worst answer is None: nobody downstream can tell “no record” from “we never asked”.

THE WRAPPER · TOOLS/CHECKS.PY

Every check returns a typed result

```
run_check(spec, app, faults)
→ CheckResult(
tool="credit_bureau",
required=True,
status=OK | UNAVAILABLE | INVALID,
data={…},
reason="timed out",
latency_ms=1.2)
```

OK

Data passed its schema check.

UNAVAILABLE

Timeout, 429, 500 or no record, with the reason.

INVALID

It answered, but the payload failed validation.

REQUIRED VS OPTIONAL

Write down what each failure does

Check	Required	If it fails (APP-001, otherwise an APPROVE)
credit_bureau	Yes	MANUAL_REVIEW · CREDIT_DATA_UNAVAILABLE
fraud_screen	Yes	MANUAL_REVIEW · CHECK_UNAVAILABLE
bank_data	No	Still APPROVE; corroboration lost (info signal)
employment_verification	No	Still APPROVE; info signal

An outage can send a case to a human. It can never turn a DECLINE into an APPROVE.

FAULT INJECTION · TOOLS/FAULTS.PY

If you can’t reproduce it, you can’t test it

```
TOOL_FAULTS="credit_bureau=timeout,bank_data=server_error"
uv run underwriter run …/APP-001.json --tool-fault fraud_screen=rate_limit
```

Five kinds

timeout · rate_limit · server_error · malformed · latency:<ms>

Per tool

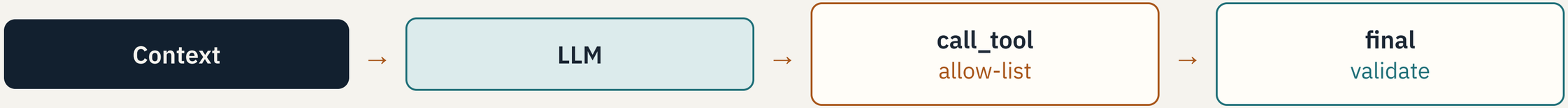
One plan names which tool misbehaves and how; the rest behave.

Never in production

A FaultPlan with faults refuses to build when APP_ENV=production.

THE AGENT LOOP · AGENTS/TOOL_AGENT.PY

Think, call a tool, read the result, repeat




```
{"action": "call_tool", "tool": "check_policy",
"arguments": {"query": "debt-to-income ratio limit"}}
<tool_result name="check_policy"> {"results": [{"id": "reg-z-1026.43-current#…", …}]}
{"action": "final", "answer": {"recommendation": "MANUAL_REVIEW", …}}
```

Tool results bring new citable ids. The final answer is validated against the context plus everything the tools returned.

THE ALLOW-LIST · AGENTS/TOOLS.PY

Tools that can read and ask, never decide

`check_policy`

Search policy and regulation; cited excerpts back.

`request_missing_document`

Queue a request to the applicant.

`get_check_result`

Read a check that already ran.

```
Tool: approve_loan({"application_id": "APP-001"}) → rejected: tool 'approve_loan' is not allowed
Tool: check_policy({"query": "debt-to-income ratio limit…"}) → ok
```

Arguments are validated with pydantic. Unknown tools and bad arguments go back to the model as tool results, not exceptions.

LIMITS

Bounded three ways, then a human

4

tool calls

A looping model is stopped at the fifth request.

30 s

deadline

Wall clock for the whole loop (AGENT_DEADLINE_S).

1

repair

One second chance for a bad reply, as in lesson 3.

Any limit hit → MANUAL_REVIEW with AGENT_LIMIT_REACHED or INVALID_MODEL_OUTPUT. The engine still decides.

LIVE WALKTHROUGH

Break the tools, then break the agent

```
uv run underwriter run …/APP-001.json --tool-fault credit_bureau=timeout
uv run underwriter run …/APP-001.json --tool-fault bank_data=server_error
uv run underwriter run …/APP-007.json
uv run underwriter run …/APP-001.json --llm-fault loop
uv run underwriter run …/APP-001.json --llm-fault disallowed_tool
uv run underwriter run …/APP-005.json --llm-fault loop
```

```
═══ APP-005: DECLINE ═══
Reasons: CREDIT_SCORE_BELOW_MIN, RECENT_DELINQUENCY, AGENT_LIMIT_REACHED
```

LAB · 100 MINUTES

Three exercises

30 MIN · FAULTS

slow_then_ok

A new fault: the first call stalls, later calls succeed. Test first. Lesson 8’s retries will need it.

40 MIN · TOOLS

get_metric

A new allow-listed tool with a pydantic argument. An unknown name comes back as an error result.

30 MIN · LIMITS

Tune the limits

Run with 1, 2 and 4 tool calls, then a 1 ms deadline. Which samples change, and why?

HOMEWORK AND INTERVIEW PREP

What strong work looks like

Criterion	Strong	Weak
Tool results	Typed status and reason	None or exceptions
Faults	On demand; off in prod	“Failed once in staging”
Required	Written down, per check	All failures alike
Allow-list	Refusals as results	Runs what the model says
Limits	Cap, deadline, fallback	Loop until it works

Interview questions

- A tool your agent needs is down. Then what?
- Stop an agent looping or running up cost
- Stop it calling a tool it shouldn’t
- Test failures you can’t reproduce
- Which checks are required, and who decided?

TAKEAWAY

Give the agent tools it can’t misuse, and a loop it can’t run forever.

Next · Lesson 5

Agentic workflows

The workflow becomes a LangGraph graph: typed state, per-node retry and timeout, and a human-review edge.